

PORL: Pretrained Offline Reinforcement Learning for the Job Shop Scheduling Problem

1st Mateo Toro Diz

*Department of Industrial Engineering
Rosenheim University
of Applied Sciences
Rosenheim, Germany
mateo.toro-diz@th-rosenheim.de*

2nd Jonathan Hoss

*Department of Industrial Engineering
Rosenheim University
of Applied Sciences
Rosenheim, Germany
jonathan.hoss@th-rosenheim.de*

3rd Noah Klarmann

*Department of Industrial Engineering
Rosenheim University
of Applied Sciences
Rosenheim, Germany
noah.klarmann@th-rosenheim.de*

Abstract—The Job Shop Scheduling Problem (JSSP) is a fundamental combinatorial optimization problem in industrial optimization. This work introduces Pretrained Offline Reinforcement Learning (PORL), a hybrid approach that combines simulation-based online pretraining with offline fine-tuning on production-specific data.

Reinforcement learning through online interaction enables exploration of general scheduling strategies, but typically relies on simulation environments and may suffer from a simulation-to-reality gap. In contrast, offline RL avoids direct interaction with the environment by learning from historical data, but its performance is strongly influenced by dataset quality and coverage. PORL combines the strengths of both paradigms by first learning a general scheduling policy through online interaction and subsequently adapting it offline to a target distribution. A KL-divergence-based policy constraint is introduced to limit deviations from the pretrained policy during fine-tuning.

The approach is evaluated on JSSP instances with distribution shift and datasets generated from heuristic, noisy-expert, and random behavioral policies. The results show that PORL consistently achieves lower optimality gaps than standalone offline RL and the considered general scheduling baselines. Furthermore, its advantage over standalone offline RL increases as dataset quality decreases, indicating reduced sensitivity to the quality and coverage of the available offline data. The results suggest that offline adaptation of pretrained policies is a promising approach for industrial scheduling environments where direct online exploration is impractical.

Index Terms—reinforcement learning, scheduling, pretraining, transfer learning, distribution shift

I. INTRODUCTION

The Job Shop Scheduling Problem (JSSP) is a classical combinatorial optimization problem in the field of operations research and management science. Solving the JSSP involves scheduling jobs and their constituent operations on available machines to minimize the total production time, commonly referred to as the makespan. JSSP is highly relevant to industrial production systems, as it serves as an idealized representation of the scheduling challenges encountered on manufacturing shop floors.

The classical JSSP defines the scheduling problem as a static instance, where the set of jobs does not change over

time. When this assumption holds, methods such as Branch & Bound [1]–[3] and Constraint Programming [4] can produce optimal or near-optimal schedules for many benchmark instances, although at a high computational cost due to the NP-hardness of the problem. However, real shop floors tend to operate in dynamic environments, where heuristic approaches are often preferred, ranging from simple Priority Dispatching Rules [5] to more sophisticated meta-heuristics [6].

Despite the success of classical approaches, their reliance on handcrafted dispatching rules or computationally intensive instance-specific optimization limits their ability to adapt to changing production environments and generalize across diverse scheduling scenarios. This has motivated the investigation of learning-based approaches, particularly Reinforcement Learning (RL), which learns scheduling policies directly from experience. More recently, Reinforcement Learning (RL) has been successfully applied to JSSP, achieving competitive scheduling performance while learning policies that can generalize across multiple problem instances [7]–[9]. However, online RL approaches require a simulation environment for training, introducing a simulation-to-reality gap.

Offline RL aims to reduce this dependency on simulation by learning directly from previously collected trajectories, eliminating the need for environment interaction during training. However, its performance remains highly dependent on the quality and coverage of the available dataset [10]–[12].

This paper introduces Pretrained Offline Reinforcement Learning (PORL), a hybrid framework that combines the exploration capabilities of online RL with the adaptability of offline RL. Pretrained Offline Reinforcement Learning (PORL) first trains a scheduling policy through online interaction with a simulated environment, enabling the agent to acquire general scheduling strategies. The resulting policy is subsequently refined using offline RL on production-specific datasets, allowing adaptation to the target shop floor without requiring online exploration. By initializing offline learning from a policy that has already explored a large portion of the scheduling state space, PORL reduces the dependence on dataset coverage and mitigates one of the key limitations of conventional offline RL.

Unlike conventional hybrid RL approaches, which employ offline pretraining followed by online fine-tuning, PORL

Supported by the Chips Joint Undertaking and its members, including top-up funding by National Authorities, within the Cynergy4MIE project (Grant Agreement No. 101140226).

adopts the reverse training order. This design is motivated by the high cost and operational risk of exploratory scheduling decisions in real-world industrial systems, where learning directly through interaction with the deployed production environment is typically infeasible.

Our work offers the following contributions:

- We introduce PORL, a novel approach combining simulation-based online pretraining with log-based offline fine-tuning.
- We propose a policy-constrained offline fine-tuning objective that adapts pretrained Q-networks using offline data while controlling divergence from the pretrained policy.
- We show that PORL achieves lower optimality gaps than both dispatching-rule baselines and standalone offline RL approaches under significant distributional shift.
- We demonstrate that PORL maintains strong performance even when trained on low-return datasets, outperforming standalone offline RL approaches.

II. RELATED WORK

A. RL for JSSP

Recent work formulates the JSSP as a sequential decision-making task, enabling the application of RL methods [7]–[9]. Most approaches employ graph-based state representations and a Graph Neural Network (GNN) to capture precedence and machine constraints, achieving strong performance and generalization on benchmark instances [7], [8], [13], [14]. Among them, Deep Q-Networks (DQN) combine temporal-difference learning with experience replay and target networks to learn scheduling policies. However, these methods are predominantly trained through online interaction with simulation environments, making them dependent on simulator fidelity and limiting their ability to leverage historical production data.

B. Offline RL for scheduling

Offline RL learns policies from fixed datasets without environment interaction, making it attractive for industrial scheduling applications where exploration is costly. A central challenge is distributional shift, which can lead to unreliable value estimates for actions not represented in the dataset [10]. Conservative Q Learning (CQL) addresses this issue through value regularization and has become a widely used offline RL algorithm [15].

The application of offline RL to JSSP remains relatively recent. Remmerden et al. [12], [16] demonstrate that CQL-based scheduling policies with action masking can achieve competitive performance when trained on datasets generated from optimal, heuristic, and noisy trajectories. Echeverria et al. [17] further explore policy-regularized offline RL for scheduling. Despite these advances, offline RL remains fundamentally constrained by dataset quality and coverage.

C. Research gap

Current RL approaches to JSSP exhibit a fundamental trade-off. Online RL enables exploration and discovery of improved

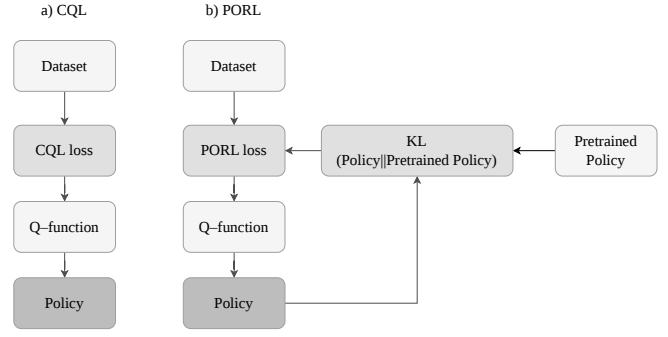


Fig. 1. Training pipeline of (a) the offline RL baseline and (b) the proposed PORL approach.

scheduling strategies but relies on simulation environments that may not accurately reflect real production systems. Offline RL leverages historical production data but is constrained by the support of the available dataset.

Hybrid RL approaches in other domains typically combine offline pretraining with online fine-tuning [18]–[20]. However, this paradigm assumes that online exploration in the target environment is feasible, which is often unrealistic in industrial scheduling due to the cost of suboptimal decisions. To the best of our knowledge, the reverse paradigm—online pretraining in simulation followed by offline adaptation to production data—has not been systematically explored for JSSP. This gap motivates the PORL framework proposed in this work.

III. ALGORITHM

Fig. 1 illustrates the overall training pipeline of the proposed method and the offline baseline.

A. Model Architecture

We use the graph representation and GNN architecture of Hoss, Link, and Klarmann [14], where each JSSP state s_t is modeled as a heterogeneous graph $G_t = (V, E)$ consisting of operation and machine nodes:

$$V = V_{\text{ops}} \cup V_{\text{mch}}. \quad (1)$$

The graph contains two edge types. Precedence edges connect consecutive operations within a job, with at most two such edges per operation. Assignment edges link each operation to its corresponding machine, with at most one assignment edge per operation. This sparse representation results in a number of edges that scales linearly with the number of operations, avoiding the quadratic edge growth of dense disjunctive graph representations.

The proposed GNN operates on the graph using a feature-based homogenization scheme. Node-type information, distinguishing operations from machines, is encoded via one-hot embeddings. This allows both node types to be processed in a shared feature space while preserving their structural distinctions.

TABLE I
GNN ARCHITECTURE HYPERPARAMETERS.

Parameter	Value
Input feature dimension	5
Hidden dimension	64
Number of GIN layers (K)	3

The resulting homogeneous graph is processed using a K -layer Graph Isomorphism Network (GIN) [21], where node embeddings are updated via multilayer perceptrons. Stacking layers enables aggregation over K -hop neighborhoods, allowing the model to capture both local scheduling constraints and global job-machine interactions.

For all experiments, we use a fixed architecture with hyperparameters listed in Table I.

B. Pretrained Offline RL

1) *Positioning*: We propose a hybrid training approach that combines online pretraining with offline fine-tuning, and name it Pretrained Offline RL (PORL). The online phase enables the agent to learn a general scheduling strategy from diverse simulated instances, while the offline phase adapts this policy to the specific distribution of target instances derived from industrial data. The fine-tuning process is constrained to prevent large deviations from the pretrained policy, thereby preserving previously learned high-quality behaviors while allowing targeted specialization.

From an industrial perspective, PORL is intended to specialize a well-trained general policy to a specific production setting using limited offline data. This work evaluates the feasibility of this adaptation principle; the online and offline training budgets are therefore kept fixed.

2) *Initialization and setup*: The PORL training process is initialized from a pretrained online DQN agent. The parameters learned during the online phase are directly transferred to the hybrid agent by loading the network weights. This transfer requires an identical Q-network architecture across both training stages. The transferred agent is then fine-tuned offline using the policy-constrained objective described below.

3) *Policy Constraint Formulation*: During offline fine-tuning, the objective is to adapt the pretrained Q-function to the distribution of the target instances while preventing large deviations from the original policy.

To achieve this, we introduce a policy-level regularization term that constrains the updated policy to remain close to the pretrained reference policy. This approach is conceptually inspired by policy-regularized offline RL methods [22], which similarly enforce proximity between the learned policy and a reference distribution—typically the behavior policy π_β —to mitigate distributional shift and ensure stable learning. In contrast, our method uses the pretrained online policy as the reference, enabling controlled adaptation while preserving previously acquired knowledge.

Since DQN is a value-based method, the policy is not learned explicitly but is implicitly defined through the Q-

values. To enable a comparison between policies, we derive a stochastic policy from the Q-function using a softmax transformation:

$$\pi(a | s) = \begin{cases} \frac{\exp\left(\frac{Q(s,a)}{\tau}\right)}{\sum_{a' \in \mathcal{A}(s)} \exp\left(\frac{Q(s,a')}{\tau}\right)}, & a \in \mathcal{A}(s), \\ 0, & a \notin \mathcal{A}(s), \end{cases} \quad (2)$$

where $Q(s,a)$ denotes the predicted action-value, τ is a temperature parameter controlling the smoothness of the distribution, and $\mathcal{A}(s)$ is the set of valid actions defined by the action mask. The reference policy π_{ref} is obtained by applying the same softmax transformation to the pretrained Q-function, and remains fixed throughout the fine-tuning process.

The deviation between the fine-tuned policy π_{new} and the pretrained reference policy π_{ref} is then quantified using the Kullback–Leibler (KL) divergence. We employ the forward KL divergence $D_{\text{KL}}(\pi_{\text{new}} \parallel \pi_{\text{ref}})$, which penalizes assigning probability mass to actions that are unlikely under the reference policy, while still allowing the updated policy to refine and reweight action preferences, thereby encouraging improvement on high-value actions while discouraging the selection of low-value ones. The resulting regularization term is defined as:

$$\mathcal{L}_{\text{KL}} = \lambda_{\text{KL}} \frac{1}{B} \sum_{i=1}^B \sum_{a \in \mathcal{A}(s_i)} \pi_{\text{new}}(a|s_i) \left[\log(\pi_{\text{new}}(a|s_i) + \varepsilon) - \log(\pi_{\text{ref}}(a|s_i) + \varepsilon) \right] \quad (3)$$

where λ_{KL} controls the strength of the constraint, B denotes the batch size, and ε is a small constant introduced for numerical stability.

In addition, we retain the CQL loss, since out-of-distribution actions remain undesirable in the proposed online-to-offline training pipeline.

The overall training objective combines the DQN loss, CQL regularization, and the KL policy constraint:

$$\mathcal{L} = \mathcal{L}_{\text{DQN}} + \mathcal{L}_{\text{CQL}} + \mathcal{L}_{\text{KL}}. \quad (4)$$

where $\mathcal{L}_{\text{DQN}}$ denotes the temporal-difference loss defined in Equation 5, while the CQL loss is defined in Equation 6.

$$\mathcal{L}_{\text{DQN}} = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} [(Q(s,a) - y)^2], \quad (5)$$

$$y = r + \gamma \max_{a'} Q(s', a')$$

$$\mathcal{L}_{\text{CQL}} = \alpha \left(\mathbb{E}_{s \sim \mathcal{D}} \left[\log \sum_a e^{Q(s,a)} \right] - \mathbb{E}_{(s,a) \sim \mathcal{D}} [Q(s,a)] \right) \quad (6)$$

This formulation limits deviations from the pretrained policy while still allowing targeted adaptation to the new data distribution, thereby enabling controlled specialization without catastrophic degradation of previously learned behavior.

IV. DATASET GENERATION

We denote the size of a JSSP instance as $J \times M$, where J is the number of jobs, and M is the number of machines.

Online pretraining is performed on standard, uniformly distributed 15×15 JSSP instances.

To emulate adaptation to a specific industrial shop floor, offline fine-tuning uses custom instance distributions that introduce a significant distribution shift.

The generated shop floor templates define (i) machine-specific processing time distributions and (ii) job precedence constraints. Processing times are sampled from machine-dependent Gaussian or uniform distributions, while 50% of the jobs are assigned partial or full precedence constraints. Partial constraints enforce only selected machine ordering relations, whereas full constraints specify a complete machine sequence for a job. The remaining jobs follow the standard unconstrained JSSP formulation. The instance size is also shifted with respect to online pretraining, with instance sizes of 10×10 .

This setup produces specialized instance distributions that differ substantially from the uniformly distributed instances commonly used during online pretraining, which were originally designed as generic optimization benchmarks. In contrast, the proposed specialized instances more closely resemble real-world shop floors, where specific product flows occur repeatedly, and machines exhibit heterogeneous characteristics. As a result, they introduce a realistic distribution shift that helps bridge the gap between simulation-based training and practical manufacturing environments.

A. Behavioral policies

To simulate scheduling decisions observed in industrial environments, three different sets of behavioral policies are introduced, ranging from random actions to exact-solver solutions, replicating previous works in the literature [12]

The first dataset is generated using two ϵ -greedy priority dispatching rules. The policies follow the corresponding heuristic while selecting a random valid action with probability ϵ , introducing variability similar to that found in real production logs.

The second set of behavioral policies is based on Remmerden, Bukhsh, and Zhang’s “Noisy-expert” policies [16]. This policy follows a schedule produced by an exact solver with probability 0.5. When not, for each scheduled action, there is a 0.1 probability that the agent will take a random valid action. When a previously scheduled random action prevents the agent from taking the scheduled optimal action, another random valid action is scheduled.

The third behavioral policy is formed by random valid actions to simulate a worst-case scenario where the production logs contain low-quality schedules.

The hyperparameters are kept constant across all 3 datasets to test the robustness of the proposed PORL algorithm to reduced-quality datasets.

TABLE II
ONLINE DQN LEARNING HYPERPARAMETERS.

Hyperparameter	Value
Learning rate	1.5×10^{-4}
Discount factor (γ)	0.99
TD loss function	smooth_l1
CQL regularization (α)	0.0
Training timesteps	250000
Replay buffer size	10000
Learning starts after	2000
Batch size	64
Training frequency (env steps/update)	4
Target network update frequency	500
ϵ (start/end/decay)	1.0/0.05/125000

V. AGENT TRAINING

This section describes the training protocols used for the online RL, standalone offline RL, and PORL agents. To enable a controlled comparison, all approaches use the same underlying network architecture, while their training data and optimization procedures differ according to the respective learning distributions, training budgets, and hyperparameter used for each approach.

A. Online RL training

The online RL baseline is trained on a fixed set of 100 15×15 uniformly distributed JSSP instances, with operation durations ranging from 0 to 100. The agent learns with the DQN algorithm, based on the architecture from section III-A with the hyperparameters listed in Table II.

B. Offline and PORL

Both the standalone offline RL baseline and the proposed PORL agent are trained on the target-specific instances described in section IV. To approximate production logs, the offline datasets are generated using the stochastic behavioral policies introduced in subsection IV-A. Each behavioral policy is rolled out on 50 instances sampled from the target distribution.

The standalone offline RL baseline is trained from scratch using CQL-DQN. In contrast, PORL initializes the Q-network with the weights obtained during online pretraining and subsequently fine-tunes the agent on the same offline dataset. This setup enables a direct comparison between learning exclusively from offline data and adapting a pretrained policy to the target distribution.

1) *Offline RL*: The offline RL algorithm is trained with fixed hyperparameters, as shown in Table III.

2) *Pretrained Offline RL*: The PORL agent is trained with the same dataset as the offline one, and is provided with the same amount of training steps, to enable a direct comparison between both approaches. The corresponding hyperparameters are shown in Table IV.

To reduce the instability inherent to the multiobjective optimization characteristic of the presented constrained DQN algorithm, the values of both the learning rate and CQL α

TABLE III
OFFLINE CQL-DQN LEARNING HYPERPARAMETERS.

Hyperparameter	Value
Learning rate	1.0×10^{-5}
Discount factor (γ)	0.99
Training timesteps	100000
TD loss function	smooth_l1
CQL regularization (α)	1.0
Batch size	64
Target network update frequency	500

TABLE IV
PRETRAINED OFFLINE AGENT LEARNING HYPERPARAMETERS.

Hyperparameter	Value
λ_{KL}	0.5
Discount factor (γ)	0.99
Training timesteps	100000
TD loss function	smooth_l1
Batch size	64
Target network update frequency	500

are changed with time. The specific ramping methods are displayed in Table V.

VI. EVALUATION PROTOCOL

To account for variability due to stochastic training, each model is trained using five independent random seeds. The same seeds are used for corresponding offline and PORL experiments to ensure consistency and comparability.

To test the generalization of the agent to new instances, the evaluation of the performance is done with a total of 100 newly generated instances, which are themselves created with 5 seeds, which also differ from the training ones. The evaluation instances are kept constant between experiments and approaches.

The performance of the trained agents is measured in terms of makespan and optimality gap. The makespan is calculated as the average makespan achieved across all evaluation instances, averaged between differently seeded experiments.

Additionally, to improve clarity, the performance is also measured in terms of the optimality gap. The optimality gap is defined as:

$$\text{Optimality Gap (\%)} = \frac{C - C^*}{C^*} \cdot 100 \quad (7)$$

where C is the makespan achieved by the evaluated agent and C^* is the optimal makespan obtained by the exact solver, with a time limit of 300 seconds.

TABLE V
PORL HYPERPARAMETERS RAMPING.

Parameter	α_{CQL}	Learning rate
Initial value	0.0	5.0×10^{-6}
Final value	1.0	2.0×10^{-6}
Ramp start	0	40000
Ramp end	40000	80000

TABLE VI
MAKESPAN OPTIMALITY GAP (%) FOR DIFFERENT SCHEDULING APPROACHES, EVALUATED ON HIGH DISTRIBUTION SHIFT INSTANCES.

Approach	Optimality gap
Generally trained PPO agent	16.19 %
Pretrained DQN agent (PORL baseline)	14.50 %
FDD/MWR	13.63 %
Offline RL agent	12.48 ± 0.63 %
Pretrained Offline RL agent	11.54 ± 0.35 %

A. Benchmarking

To validate the performance of both the offline and hybrid agents, there are several benchmarks against which the performance is to be compared.

By the use of the optimality gap metric, a comparison against the optimal solver is already implicit. Additionally, we also compare the results against the results achieved by:

- Priority Dispatching Rules
- A state-of-the-art PPO agent trained on benchmark, uniformly distributed 20×20 JSSP instances, following the work of Hoss, Link, and Klarmann [14].

VII. RESULTS

The results in Table VI show that the proposed hybrid PORL setup achieves lower optimality gaps when compared to both the standard offline RL approach and commonly used general solutions to the JSSP, including the Flow Due Date / Most Work Remaining (FDD/MWR) priority dispatching rule and an online-trained RL agent [14]). The results displayed in Table VI are averaged across all 5 training seeds, which remain the same for both offline and PORL.

The proposed PORL pipeline achieves a relative reduction of 7.5% in the optimality gap when compared to classical offline RL, and 28.7% when compared to a state-of-the-art PPO agent trained on traditional, uniformly distributed JSSP instances.

The standard deviation of the optimality gap across seeds is substantially smaller than the observed performance difference between Offline RL and PORL, and at the same time, slightly smaller than the standard deviation of Offline RL. The distribution of the optimality gap for both PORL and offline RL can be seen in Fig. 2.

To evaluate the robustness of the proposed pipeline against varying dataset quality, the agents are trained on three different datasets, each generated by one of the behavioral policies explained in section IV-A. The results in Table VII show the effect of using heuristics, noisy-expert, and purely random datasets on both traditional offline RL and our proposed PORL agent. The results in 2 show how dataset quality affects both approaches.

The proposed PORL agent outperforms the standard offline CQL baseline across all dataset configurations. The performance gap increases as the quality of the behavioral policy decreases, particularly for the noisy-expert and fully random datasets.

TABLE VII
MAKESPAN OPTIMALITY GAP (%) FOR OFFLINE AND PORL AGENTS WITH DIFFERENT DATASET COMPOSITIONS.

Approach	Combined Heuristics	Noisy-Expert	Random
Offline RL agent	12.48 ± 0.63 %	16.89 ± 2.15 %	26.60 ± 1.24 %
Pretrained Offline RL agent	11.54 ± 0.35 %	13.19 ± 0.88 %	21.08 ± 1.74 %

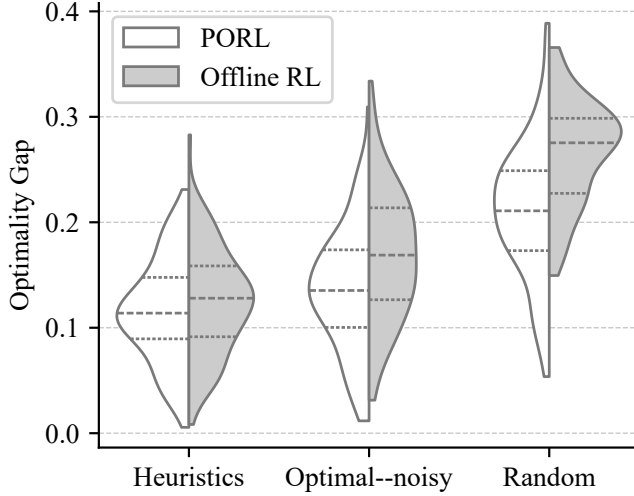


Fig. 2. PORL and Offline RL optimality gap comparison

VIII. DISCUSSION AND LIMITATIONS

The results demonstrate that the proposed PORL framework achieves lower optimality gaps than both the standalone offline RL baseline and the considered online RL approaches under substantial distributional shift. This suggests that knowledge acquired through online interaction with a simulator can be successfully transferred to an offline adaptation setting, despite differences between the source and target instance distributions.

A possible explanation for this improvement is that online pretraining exposes the agent to a broader range of scheduling states and actions than those contained in the offline dataset. As a result, the pretrained network provides more informative initial value estimates and state representations before offline optimization begins. In contrast, conventional offline RL must learn these representations exclusively from the available dataset, making it more sensitive to limitations in dataset coverage and quality. The observed performance gains indicate that the pretrained policy acts as a useful prior that guides learning towards high-quality scheduling strategies while still allowing adaptation to the target distribution.

The results further show that the FDD/MWR dispatching rule outperforms both online RL agents trained on uniformly distributed benchmark instances. This suggests that, in the evaluated setting, distributional shift may have a larger impact on performance than policy complexity. While the online RL agents learn sophisticated scheduling strategies for the source distribution, these strategies do not fully transfer to

the target shop floor. In contrast, dispatching rules are largely distribution-agnostic and therefore exhibit greater robustness under changing operating conditions. Nevertheless, both of-line RL and PORL outperform the heuristic baseline, highlighting the benefit of adapting policies to production-specific data.

The reduced sensitivity to dataset quality observed in PORL aligns with the broader offline RL literature, which identifies dataset quality and coverage as a major influence on the performance [10], [11]. The results indicate that limiting divergence from the online pretrained policy contributes to maintaining PORL’s superior performance with respect to Priority Dispatching Rules for 2 of the 3 datasets, while the performance from offline RL degrades more rapidly as the dataset quality does.

Despite its superior performance, PORL exhibits greater training instability than conventional offline RL. A likely explanation is the interaction between multiple optimization objectives. During fine-tuning, the DQN loss promotes value estimation, the CQL objective encourages conservative behavior with respect to the offline dataset, and the KL regularization term constrains deviations from the pretrained policy. When the pretrained policy and the behavioral policy underlying the dataset differ substantially, these objectives may provide conflicting optimization signals, leading to less stable training dynamics. In our experiments, this effect was partially mitigated through loss scheduling and hyperparameter ramping, although additional tuning complexity is introduced compared to standard offline RL.

While a full ablation study was outside the scope of this work, preliminary experiments without KL regularization resulted in catastrophic forgetting and substantially degraded performance, with optimality gaps reaching 25%, indicating that the constraint plays an important role in preserving transferred knowledge.

The scope of the present evaluation is deliberately limited. First, the experiments are restricted to synthetic 10×10 JSSP instances and only a single distribution-shift scenario. While the results indicate that PORL can effectively adapt to specialized instance distributions, further validation on larger scheduling problems and real production datasets is required. Second, the proposed approach relies on the availability of a suitable simulation environment for pretraining. Significant mismatch between simulation and deployment conditions may reduce the effectiveness of the transferred policy. Finally, the impact of the KL regularization strength and alternative policy-constraining mechanisms remains an open research question and warrants further investigation.

IX. CONCLUSION

This paper introduced PORL, a hybrid training framework that combines simulation-based online RL with offline adaptation to production-specific scheduling data. Unlike conventional hybrid RL approaches that rely on offline pretraining followed by online fine-tuning, PORL reverses this paradigm by first learning a general scheduling policy through online interaction and subsequently adapting it using offline RL. To enable controlled adaptation, we further introduced a KL-divergence-based policy constraint that limits deviations from the pretrained policy during offline fine-tuning.

Experiments on JSSP instances under substantial distributional shift demonstrated that PORL consistently outperforms standalone offline RL, online RL baselines, and priority dispatching rules. The results indicate that online pretraining provides informative prior knowledge that improves adaptation to specialized scheduling environments, reducing the dependence of offline RL on dataset quality and coverage.

These findings suggest that combining online exploration with offline adaptation is a promising direction for industrial scheduling applications where direct online learning is impractical, and standalone offline learning can be sensitive to the dataset quality and coverage.

Future work will focus on evaluating PORL on larger-scale scheduling problems and real-world production datasets, as well as investigating different levels of distribution shift and their impact on PORL performance.

REFERENCES

- [1] P. Brucker, B. Jurisch, and B. Sievers, "A Branch and Bound Algorithm for the Job-Shop Scheduling Problem," *Discrete Applied Mathematics*, vol. 49, no. 1, pp. 107–127, 1994.
- [2] C. Artigues and D. Feillet, "A Branch and Bound Method for the Job-Shop Problem with Sequence-Dependent Setup Times," *Annals of Operations Research*, vol. 159, pp. 135–159, Mar. 2008.
- [3] E. B. Nababan, A. R. Hamdan, S. Abdullah, and M. S. Zakaria, "Branch and Bound Algorithm in Optimizing Job Shop Scheduling Problems," in *2008 International Symposium on Information Technology*, vol. 1, pp. 1–5, Aug. 2008.
- [4] G. Da Col and E. C. Teppan, "Industrial-Size Job Shop Scheduling with Constraint Programming," *Operations Research Perspectives*, vol. 9, p. 100249, Jan. 2022.
- [5] M. Habib Zahmani, B. Atmani, A. Bekrar, and N. Aissani, "Multiple Priority Dispatching Rules for the Job Shop Scheduling Problem," in *Proceedings of the 3rd International Conference on Control, Engineering and Information Technology*, May 2015.
- [6] D. C. Hajariwala, S. S. Patil, and S. M. Patil, "A Review of Metaheuristic Algorithms for Job Shop Scheduling," *Engineering Access*, vol. 11, no. 1, pp. 65–91, 2025.
- [7] C. Zhang, W. Song, Z. Cao, J. Zhang, P. S. Tan, and C. Xu, "Learning to Dispatch for Job Shop Scheduling via Deep Reinforcement Learning," Oct. 2020. arXiv:2010.12367.
- [8] J. Park, J. Chun, S. H. Kim, Y. Kim, and J. Park, "Learning to Schedule Job-Shop Problems: Representation and Policy Learning Using Graph Neural Networks and Reinforcement Learning," *International Journal of Production Research*, vol. 59, pp. 3360–3377, June 2021. arXiv:2106.01086.
- [9] R. Maharjan, P.-A. Andersen, and L. Jiao, "A Reinforcement Learning Environment for Job Shop Scheduling with Tool Management," in *Artificial Intelligence XLII* (M. Bramer and F. Stahl, eds.), pp. 80–93, Springer Nature Switzerland, 2026.
- [10] S. Levine, A. Kumar, G. Tucker, and J. Fu, "Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems," Nov. 2020. arXiv:2005.01643.
- [11] K. Schweighofer, M.-c. Dinu, A. Radler, M. Hofmarcher, V. P. Patil, A. Bitto-nemling, H. Eghbal-zadeh, and S. Hochreiter, "A Dataset Perspective on Offline Reinforcement Learning," in *Proceedings of The 1st Conference on Lifelong Learning Agents*, pp. 470–517, PMLR, Nov. 2022.
- [12] J. v. Remmerden, Z. Bukhsh, and Y. Zhang, "Generalizing Beyond Sub-optimality: Offline Reinforcement Learning Learns Effective Scheduling through Random Data," Sept. 2025. arXiv:2509.10303.
- [13] S. Munikoti, D. Agarwal, L. Das, M. Halappanavar, and B. Natarajan, "Challenges and Opportunities in Deep Reinforcement Learning with Graph Neural Networks: A Comprehensive Review of Algorithms and Applications," Nov. 2022. arXiv:2206.07922.
- [14] J. Hoss, M. Link, and N. Klarmann, "Scalable Production Scheduling: Linear Complexity via Unified Homogeneous Graphs," Apr. 2026. arXiv:2604.23841.
- [15] A. Kumar, A. Zhou, G. Tucker, and S. Levine, "Conservative Q-Learning for Offline Reinforcement Learning," Aug. 2020. arXiv:2006.04779.
- [16] J. v. Remmerden, Z. Bukhsh, and Y. Zhang, "Offline Reinforcement Learning for Learning to Dispatch for Job Shop Scheduling," *Machine Learning*, vol. 114, July 2025.
- [17] I. Echeverria, M. Murua, and R. Santana, "Offline Reinforcement Learning for Job-Shop Scheduling Problems," *Applied Soft Computing*, vol. 184, p. 113736, Dec. 2025.
- [18] A. Nair, A. Gupta, M. Dalal, and S. Levine, "AWAC: Accelerating Online Reinforcement Learning with Offline Datasets," Apr. 2021. arXiv:2006.09359.
- [19] Z. Yao, H.-S. Yoon, and Y.-K. Hong, "Control of Hybrid Electric Vehicle Powertrain Using Offline-Online Hybrid Reinforcement Learning," *Energies*, vol. 16, p. 652, Jan. 2023.
- [20] J. Chen and N. Luo, "An Offline-to-Online Reinforcement Learning Framework with Trajectory-Guided Exploration for Industrial Process Control," *Journal of Process Control*, vol. 154, p. 103535, Oct. 2025.
- [21] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How Powerful are Graph Neural Networks?," Feb. 2019. arXiv:1810.00826.
- [22] S. Fujimoto, D. Meger, and D. Precup, "Off-Policy Deep Reinforcement Learning without Exploration," Aug. 2019. arXiv:1812.02900.